Acceso a Datos - Semana 5

UD1. Manejo de ficheros - XML básico e introducción a JSON

Objetivo: conocer XML a un nivel esencial y dedicar el resto de la semana a comprender la estructura de JSON y su relación con objetos Java.

Cuando queremos guardar datos que puedan ser leídos por diferentes aplicaciones y plataformas, lo más adecuado es hacer uso de formatos estándares. Los lenguajes de marcas es un caso concreto y el más conocido es el XML (eXtensible Markup Language).

1. XML: idea general

XML es un lenguaje de marcado para representar datos estructurados mediante etiquetas. En este curso se trabajará únicamente lo necesario para leer, recorrer y generar documentos sencillos.

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
    <alumno id="1">
        <nombre>Ana</nombre>
        <edad>22</edad>
    </alumno>
</alumnos>
```

Reglas básicas de XML

- Debe existir un único elemento raíz.
- Las etiquetas deben cerrarse y estar correctamente anidadas.
- XML diferencia mayúsculas de minúsculas.
- Puede contener atributos y elementos.
- Un documento sintácticamente correcto se denomina bien formado.

2. Analizadores XML

Un parser o analizador XML es una clase que permite analizar un fichero XML y extraer información de él, relacionándola según su posición en la jerarquía. Los analizadores, según su forma de funcionar, pueden ser:

- **Analizadores secuenciales o sintácticos**: Van extrayendo el contenido según se van descubriendo los hashtags de apertura y cierre. Son rápidos, pero tienen que leer todo el documento para acceder a una parte concreta. En Java existe el SAX como analizador secuencial
- **Analizadores Jerarquicos**: Son los más utilizados, ya que guardan todos los datos del documento XML en memoria, en forma de estructura jerarquizada (DOM o modelo de objetos del documento).

3. Acceso a Ficheros XML con DOM

Para poder trabajar con DOM en Java necesitamos las clases e interfaces que componen el paquete org.w3c.com y el paquete javax.xml.parser del API estándar de Java.

Las clases **DocumentBuilderFactory** y **DocumentBuider** ofrecen métodos para cargar documentos desde una fuente de datos.
Algunas de las interfaces que se tienen son:

- Document: Objeto que equivale al documento XML. Permite crear nuevos nodos en el documento.
- Element: Representa al documento XML.
- Node: Representa un nodo del documento.
- NodeList: Contiene una lista de nodos
- Attr: Contiene los atributos de un nodo
- Text: Datos de un elemento (texto)
- CharacterData: Represetna los datos carácter del documento
- DocumentType: Información de la etiqeueta DOCTYPE

4. Lectura básica con DOM

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new File("alumnos.xml"));

NodeList lista = doc.getElementsByTagName("alumno");
```

DOM carga el documento completo en memoria y lo representa como un árbol. Para esta unidad es suficiente trabajar con documentos pequeños.

5. Recorrer elementos

```
for (int i = 0; i < lista.getLength(); i++) {
    Element alumno = (Element) lista.item(i);
    String id = alumno.getAttribute("id");
    String nombre = alumno.getElementsByTagName("nombre")
                          .item(0).getTextContent();
    System.out.println(id + " - " + nombre);
}
```

6. Creación de Fichero XML con DOM

```
public static void main(String[] args) {
    int codigo[] = {1,2,3,4,5,6,7};
    String apellido[] = { "FERNANDEZ", "GIL", "LOPEZ", "RAMOS", "SEVILLA", "CASILLA", "REY"};
    short dep[] = {10, 20, 10, 10, 30, 30, 20};
    Double salario[] = {1000.50, 2400.50, 2000.0, 1650.50, 950.0, 950.50, 2000.50};

    //Creamos la factoría para poder generar nuevos documentos XML
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();

    try {
        DocumentBuilder builder = factory.newDocumentBuilder();
        DOMImplementation implementation = builder.getDOMImplementation();
        //Creamos el documento XML vacio
        Document document = implementation.createDocument(null, "Empleados", null);
        document.setXmlVersion("1.0"); //Asignamos una versión al XML

        for (int i=0;i<apellido.length;i++) {
            //creamos el nodo empleado
            Element empleado= document.createElement("empleado");
            //agregamos el nodo a la raiz
            document.getDocumentElement().appendChild(empleado);
            //creamos los elementos hijo del nodo empleado mediante la función CrearElemento
            crearElementoEmpleado("id",Integer.toString(codigo[i]), empleado, document);
            crearElementoEmpleado("apellido",apellido[i], empleado, document);
            crearElementoEmpleado("dep",Integer.toString(dep[i]), empleado, document);
            crearElementoEmpleado("salario",Double.toString(salario[i]), empleado, document);
        }

        //Generamos el fichero XML a partir del documento creado
        Source source = new DOMSource(document);
        Result result = new StreamResult(new java.io.File("Empleados.xml"));
        Transformer transformer = TransformerFactory.newInstance().newTransformer();
        transformer.transform(source, result);
    } catch (TransformerException e) {
        e.printStackTrace();
    } catch (ParserConfigurationException e) {
        e.printStackTrace();
    }
}
```

Los imports:

```
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.Result;
import javax.xml.transform.Source;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

import org.w3c.dom.DOMImplementation;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Text;
```

El método para dotar de contenido al empleado:

```
private static void crearElementoEmpleado(String datoEmpleado, String valor, Element empleado, Document documento) {
    Element elem = documento.createElement(datoEmpleado);
    Text text = documento.createTextNode (valor);
    elem.appendChild(text);
    empleado.appendChild(elem);
}
```

Lectura de fichero XML con DOM:

```
import java.io.File;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;

public class LeerEmpleadosDOM {

    public static void main(String[] args) {
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        try {
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document document = builder.parse(new File("Empleado.xml"));
            document.getDocumentElement().normalize();

            System.out.printf("Elemento raiz: %s %n", document.getDocumentElement().getNodeName());
            //creamos una lista con todos los nodos "empleado"
            NodeList empleados = document.getElementsByTagName("empleado");
            System.out.printf("Nodos empleado a recorrer: %d %n", empleados.getLength());

            //recorrer la lista
            for (int i =0; i<empleados.getLength();i++) {
                Node emp= empleados.item(i);
                if (emp.getNodeType()==Node.ELEMENT_NODE) {//validamos el tipo de nodo y obtenemos los elementos
                    Element elemento = (Element) emp;
                    System.out.printf("ID = %s %n", elemento.getElementsByTagName("id").item(0).getTextContent());
                    System.out.printf("ID = %s %n", elemento.getElementsByTagName("apellido").item(0).getTextContent());
                    System.out.printf("ID = %s %n", elemento.getElementsByTagName("dep").item(0).getTextContent());
                    System.out.printf("ID = %s %n", elemento.getElementsByTagName("salario").item(0).getTextContent());
                }
            }
        }catch (Exception e){
            e.printStackTrace();
        }
    }

}
```

7. JSON: estructura fundamental

JSON representa datos mediante pares clave-valor y arrays. Es muy utilizado en aplicaciones modernas y APIs.

```
{
  "id": 1,
  "nombre": "Ana",
  "edad": 22,
  "activo": true
}
```

8. Tipos de valores JSON

- string
- number
- boolean
- null
- object
- array

9. Arrays y objetos anidados

```
{
  "curso": "2DAM",
  "alumnos": [
    {"id": 1, "nombre": "Ana"},
    {"id": 2, "nombre": "Pedro"}
  ]
}
```

10. XML frente a JSON

XML	JSON
Etiquetas de apertura/cierre	Claves y valores
Más verboso	Normalmente más compacto
Atributos y elementos	Objetos y arrays
Muy usado en configuración/documentos	Muy usado en APIs y aplicaciones web
Procesamiento con DOM/SAX/etc.	Mapeo muy natural a objetos

11. Preparación para Java + JSON

En la semana siguiente se utilizará una librería como Gson para transformar objetos Java en JSON y viceversa. La idea central será la serialización en un formato independiente de Java.

12. Al terminar la semana debes saber

- Reconocer y crear un XML sencillo y bien formado.
- Leer elementos y atributos básicos con DOM.
- Reconocer objetos, arrays y tipos JSON.
- Interpretar estructuras JSON anidadas.
- Comparar XML y JSON sin necesidad de profundizar en APIs XML avanzadas.